

IES JULIO VERNE

BARGAS (TOLEDO)

Ciclo Formativo de Grado Superior

DESARROLLO DE APLICACIONES WEB (DAW)

MEMORIA DE PRÁCTICAS

Tema 3 . Práctica 1 - Excepciones

Alumno: Dayra Daniela Gallegos Velasquez

Ariadna Lilibeth Montero Matínez

Adrián Toledano López-Gálvez

Asignatura: Desarrollo de Aplicaciones en Entorno Servidor

Profesor/a: Mario Marugán

Curso académico: 2026 – 2027

Fecha: 02/11/2026

CONTENIDO

1. ENLACE A GITHUB 2
2. OBJETIVOS 2
3. DESARROLLO DE LA PRÁCTICA 2
4. RESULTADOS OBTENIDOS 15
5. DIFICULTADES ENCONTRADAS Y SOLUCIONES APLICADAS 19
6. CONCLUSIONES 19

ENLACE A GITHUB

https://github.com/ariadnamonterom-sudo/P3-GA1.git

OBJETIVOS

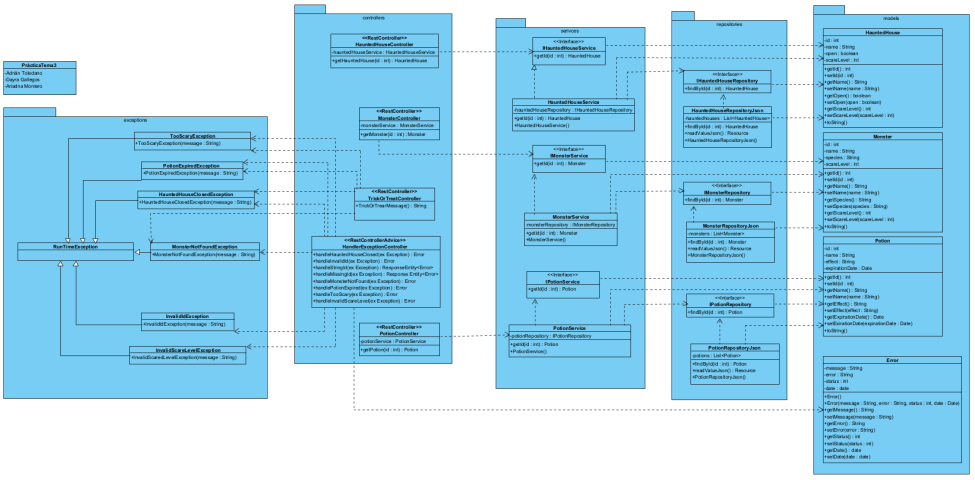
- Diseño y la manipulación de objetos
- Usar el patrón **MVC** para separar la lógica de negocio, presentación y datos.
- Detección y modelado de relaciones entre clases
- Uso de anotaciones de Spring MVC.
- Inyección de dependencias.
- Aplicación de todos los conocimientos sobre programación aprendidos hasta el momento.
- Uso de modificadores de visibilidad
- Practicar el uso de excepciones personalizadas.
- Aprender a capturar y gestionar errores.
- Devolver respuestas JSON informativas y con código de estado HTTP apropiado.
- Uso de patrones de desarrollo.

1. DESARROLLO DE LA PRÁCTICA

En esta práctica se desarrollan los diferentes endpoints de una API REST dedicada a gestionar monstruos, pociones y casas encantadas. Conecta controladores, servicios y modelos, centrándose en el control de errores mediante excepciones personalizadas y un manejador global.

Para ello, se han utilizado los conocimientos aprendidos en el Tema 3 de Desarrollo Web en Entorno Servidor, concretamente el tema orientado a "Excepciones".

1. Diseño



Para este diagrama se ha utilizado el software Visual Paradigm para poder desarrollarlo con la mayor precisión posible.

Cuenta con 5 carpetas interconectadas entre sí: ‘controllers’, ‘models’, ‘services’, ‘repositories’, y ‘exceptions’. La carpeta Controllers cuenta con cinco archivos de java que sirven de nexo entre los servicios y por otro lado el controlador HandlerExceptionContoller sirve de conexión con la carpeta donde se guardan las excepciones personalizadas.

La carpeta de Models incluye las clases puras a las que tienen que acceder los Servicios y los Repositorios, en este caso se cuenta con ‘Monster’, ‘Potion’ y ‘HauntedHouse’ que recoge todos los atributos y métodos de los que están formadas y que luego son utilizados para la instancia de los objetos de dichas clases. .

Se cuenta también con la carpeta ‘repositories’ que como su nombre indica guarda todos los repositorios de cada modelo y una interfaz para cada uno de ellos. Están conectados directamente con los servicios y realizan la función de acceder a los archivos.json para poder leer las instancias y poderlas mostrar al interactuar con el endPoint correspondiente.

Por otro lado está la carpeta ‘services’ que guarda todos un servicio por modelo creado y una interfaz por cada servicio en la que se declaran los métodos. Dentro de cada servicio se reescribe el método de la interfaz donde se desarrolla toda la lógica del manejo de las excepciones que tienen que activarse cuando se introduce un valor de id en las URLs.

Y por último, se cuenta con la carpeta ‘exceptions’ y en ella se han creado las 4 excepciones necesarias para el manejo de errores según el enunciado de esta práctica Además se han creado otras dos más para manejar otro tipo de

errores que se han ido presentando durante la realización del ejercicio. Todas estas excepciones, para su correcto funcionamiento, heredan directamente de una clase superior (padre) RunTimeException.

2. Código fuente (Enunciado de la práctica)

La práctica consta de una API REST para la gestión de monstruos, pociones mágicas y casas encantadas. Para ello, se ha utilizado 'Spring', implementando excepciones personalizadas y un manejador global para devolver respuestas en formato JSON.

El enunciado proporcionado es:

El Ayuntamiento de Transilvania ha encargado a tu equipo de desarrolladores la creación de una API REST para gestionar monstruos, pociones mágicas y casas encantadas.

Todo parece sencillo… hasta que los errores comienzan a aparecer. Monstruos que desaparecen, pociones que explotan, casas embrujadas que cierran sin aviso...

Tu misión es crear un sistema que no sólo funcione, sino que maneje con elegancia y humor todos esos horrores a través de excepciones personalizadas y un manejador global de errores.

A continuación, se indican los modelos a utilizar:

```java
public class Error {
    private String message;
    private String description;
    private int status;
    private Date timestamp;

    public Error() {
    }

    public Error(String message, String description, int status, Date timestamp) {
        this.message = message;
        this.description = description;
        this.status = status;
        this.timestamp = timestamp;
    }
```

```java
public class Monster {
    private int id;
    private String name;
    private String species;
    private int scareLevel;


    public Monster() {
    }

    public Monster(int id, String name, String species, int scareLevel) {
        this.id = id;
        this.name = name;
        this.species = species;
        this.scareLevel = scareLevel;
    }
```

```java
public class Potion {
    private int id;
    private String name;
    private String effect;
    private Date expirationDate;

    public Potion() {

    }

    public Potion(int id, String name, String effect, Date expirationDate) {
        this.id = id;
        this.name = name;
        this.effect = effect;
        this.expirationDate = expirationDate;
    }
```

```java
public class HauntedHouse {
    private int id;
    private String name;
    private boolean open;
    private int scareLevel;

    public HauntedHouse() {
    }

    public HauntedHouse(int id, String name, boolean open, int scareLevel) {
        this.id = id;
        this.name = name;
        this.open = open;
        this.scareLevel = scareLevel;
    }
```

Repositories:

```java
public interface IMonsterRepository {
    public Monster findById(int id);
}
```

```java
@Repository
public class MonsterRepositoryJson  implements  IMonsterRepository{

    //Creamos la lista de monstruos que guardará los datos json.
    private List<Monster> monsters = new ArrayList<>();

    //El constructor busca el archivo json de monstruo.
    public MonsterRepositoryJson() {
        Resource resource = new ClassPathResource("json/monsters.json");
        readValueJson(resource);
    }

    //Método que lee el archivo json y lo convierte en objetos Java.
    private void readValueJson(Resource resource) {
        ObjectMapper objectMapper = new ObjectMapper();
        try {
            //Convierte el json en una lista de monstruos y lo pasa a una Lista.
            monsters = Arrays.asList(objectMapper.readValue(resource.getInputStream(), valueType: Monster[].class));

        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    @Override
    public Monster findById(int id) {
        return monsters.stream().filter((m -> m.getId() == id))//Filtra los monstruos que coincidan en id
            .findFirst().orElse(other: null);//Muestra el primero que encuentra sino, devuelve nulo.
    }
}
```

```java
public interface IPotionRepository {
    public Potion findById(int id);
}
```

```java
@Repository
public class PotionRepositoryJson implements IPotionRepository{

    //Creamos la lista de monstruos que guardará los datos json.
    private List<Potion> potions = new ArrayList<>();

    //El constructor busca el archivo json de pociones
    public PotionRepositoryJson() {
        Resource resource = new ClassPathResource("json/potions.json");
        readValueJson(resource);
    }

    //Método que lee el archivo json y lo convierte en objetos Java
    private void readValueJson(Resource resource) {
        ObjectMapper objectMapper = new ObjectMapper();
        try {
            //Convierte el Json en un array de pociones y lo pasa a una Lista
            potions = Arrays.asList(objectMapper.readValue(resource.getInputStream(), valueType: Potion[].class));

        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    @Override
    public Potion findById(int id) {
        return potions.stream().filter(p -> p.getId() == id)//Filtra las pociones que contengan el mismo id.
            .findFirst().orElse(other: null);//Devuelve el primero que encuentra sino, devuelve null.
    }
}
```

```java
public interface IHauntedHouseRepository {
    public HauntedHouse findById(int id);
}
```

```java
@Repository
public class HauntedHouseRepositoryJson implements IHauntedHouseRepository{

    //Creamos una lista de casas para donde se guardaran los datos del json.
    private List<HauntedHouse> hauntedHouses = new ArrayList<>();

    //El constructor busca el archivo json
    public HauntedHouseRepositoryJson() {
        Resource resource = new ClassPathResource("json/houses.json");
        readValueJson(resource);
    }

    //Método que lee el archivo json y lo convierte en objetos Java
    private void readValueJson(Resource resource) {
        ObjectMapper objectMapper = new ObjectMapper();
        try {
            //Convierte el Json en un array de casas y lo pasa a una Lista
            hauntedHouses = Arrays.asList(objectMapper.readValue(resource.getInputStream(), valueType: HauntedHouse[].class));

        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    @Override
    public HauntedHouse findById(int id) {
        return hauntedHouses.stream().filter(h -> h.getId() == id) //Filtra las casa que contengan el mismo id
            .findFirst().orElse(other: null); //Elige la primera, si no la encuentra devuelve null.
    }
}
```

Services:

```java
public interface IMonsterService {
    public Monster findById(int id);
}
```

```java
@Service
public class MonsterService implements IMonsterService{

    private IMonsterRepository monsterRepository;

    public MonsterService(IMonsterRepository monsterRepository) {
        this.monsterRepository = monsterRepository;
    }

    //Se sobreescribe el método añadido en la interfaz
    @Override
    public Monster findById(int id) {
        Monster monster = monsterRepository.findById(id);

        //Si el id es nulo salta la excepción previamente pedida.
        if(monster == null) {
            throw new MonsterNotFoundException("El monstruo con ID " + id + " no existe.");
        }

        //Excepción añadida: Si el nivel de miedo es menor de 0 o mayor que 10 salta la excepción.
        if(monster.getScareLevel() < 0 || monster.getScareLevel() > 10 ) {
            throw new InvalidScareLevelException(message: "Nivel de miedo no válido");
        }

        return monster;
    }
}
```

```java
public interface IPotionService {
    public Potion findById(int id);
}
```

```java
@Service
public class PotionService implements IPotionService{

    private IPotionRepository potionRepository;

    public PotionService(IPotionRepository potionRepository) {
        this.potionRepository = potionRepository;
    }

    //Se sobreescribe el método añadido previamente en la interfaz
    @Override
    public Potion findById(int id) {
        Potion potion = potionRepository.findById(id);

        //Si la pocion no existe salta la excepción añadida "InvalidIdException"
        //Controla las pociones con id inválido
        if (potion == null) {
            throw new InvalidIdException(message: "Poción no encontrada");
        }

        //Se crea la excepción pars comprobar la fecha de caducidad
        // Por un lado comprobamos que el dato de la fecha no esté en nulo
        // y por el otro se compueba que la fecha tenga un valor superior al actual del aquí y el ahora.
        if(potion.getExpirationDate() != null && potion.getExpirationDate().before(new Date())) {
            throw new PotionExpiredException("La poción '" + potion.getName() + "' caducó el " + potion.getExpirationDate());
        }

        return potion;
    }
}
```

```java
public interface IHauntedService {
    public HauntedHouse findById(int id);
}
```

```java
@Service
public class HauntedHouseService implements IHauntedService{

    private IHauntedHouseRepository hauntedHouseRepository;

    public HauntedHouseService(IHauntedHouseRepository hauntedHouseRepository) {
        this.hauntedHouseRepository = hauntedHouseRepository;
    }

    //Se sobreescribe el método añadido previamente en la interfaz
    @Override
    public HauntedHouse findById(int id) {
        HauntedHouse hauntedHouse = hauntedHouseRepository.findById(id);

        //Si la pocion no existe salta la excepción añadida "InvalidIdException"
        //Controla las pociones con id inválido
        if (hauntedHouse == null) {
            throw new InvalidIdException(message: "Haunted House no encontrado.");
        }

        //Excepcion para controlar el nivel de miedo "InvalidScareLevelException"
        if(hauntedHouse.getScareLevel() < 0 || hauntedHouse.getScareLevel() > 10 ) {
            throw new InvalidScareLevelException(message: "Nivel de miedo no válido");
        }

        //Si Haunted house esta cerrado (false), salta la exception "HauntedHouseClosedException"
        if(hauntedHouse.getOpen() == false) {
            throw new HauntedHouseClosedException(message: "Hautend House esta fuera del horario de sustos");
        }

        //Si el nivel de miedo es mayor a 8, "TooScaryException" salta por demasiado aterradora
        if (hauntedHouse.getScareLevel() > 8) {
            throw new TooScaryException(message: "Experiencia demasiado aterradora");
        }

        return hauntedHouse;
    }
}
```

Exceptions:

```java
public class MonsterNotFoundException extends RuntimeException {
    public MonsterNotFoundException(String message) {
        super(message);
    }
}
```

```java
public class PotionExpiredException extends RuntimeException{
    public PotionExpiredException(String message) {
        super(message);
    }
}
```

```java
public class HauntedHouseClosedException extends RuntimeException{
    public HauntedHouseClosedException(String message) {
        super(message); //Envía el mensaje a la clase padre (RuntimeException)
    }

}
```

```java
public class TooScaryException extends RuntimeException{
    public TooScaryException(String message) {
        super(message);
    }
}
```

```java
public class InvalidScareLevelException extends RuntimeException{
    public InvalidScareLevelException(String message) {
        super(message);
    }
}
```

```java
public class InvalidIdException extends RuntimeException{
    public InvalidIdException(String message) {
        super(message);
    }
}
```

Controller:

```java
@RestController
@RequestMapping("/monsters")
public class MonsterController {

    @Autowired     Convert @Autowired field into Constructor Parameter
    private IMonsterService monsterService;

    @GetMapping("/{id}")
    public Monster getMonster(@PathVariable(required = false) int id) {
        return monsterService.findById(id);
    }
}
```

```java
@RestController
@RequestMapping("/potion")
public class PotionController {

    @Autowired     Convert @Autowired field into Constructor Parameter
    private IPotionService potionService;

    @GetMapping("/{id}")
    public Potion getPotion(@PathVariable int id) {
        return potionService.findById(id);
    }
}
```

```java
@RestController
@RequestMapping("/enter")
public class HauntedHouseController {

    @Autowired     Convert @Autowired field into Constructor Parameter
    private IHauntedService hauntedService;

    @GetMapping("/{id}")
    public HauntedHouse getHauntedHouse(@PathVariable int id) {
        return hauntedService.findById(id);
    }
}
```

```java
@RestController
public class TrickOrTreatController {

    @GetMapping("/trickortreat")
    public String TrickOrTreatMessage() {

        //Random para elegir entre 2 opciones:
            //1. Truco o trato
            //2. Exception aleatoria
        Random randomOption = new Random();

        int option = randomOption.nextInt(bound: 2);//2 posibilidades

        //Si la opción es 0 devolvemos el mensaje "Truco o trato" al usuario
        if(option == 0) {
            return "Truco o trato!";
        } else {
            //Si no, con un segundo random se elige un número del 0 al 3
            Random randomException = new Random();
            int exception = randomException.nextInt(bound: 4);

            //Dependiendo del número, lanza una excepción u otra.
            switch (exception) {
                case 0:
                    throw new MonsterNotFoundException(message: "¡Susto! El monst
                case 1:
                    throw new PotionExpiredException(message: "¡Puaj! Esta pociór
                case 2:
                    throw new HauntedHouseClosedException(message: "¡Demasiado ta
                case 3:
                    throw new TooScaryException(message: "¡ERROR 666! Los niveles
                default:
                    return "Truco o trato!";
            }
        }
    }

}
```

```java
@RestControllerAdvice
public class HandlerExceptionController {

    @ExceptionHandler(MonsterNotFoundException.class)
    public Error handlerMonsterNotFound(MonsterNotFoundException ex) {
        return new Error(ex.getMessage(), description: "Monstruo no encontrado",
        HttpStatus.BAD_REQUEST.value(), new Date());
    }

    @ExceptionHandler(PotionExpiredException.class)
    public Error handlerPotionExpired(PotionExpiredException ex) {
        return new Error(ex.getMessage(), description: "La poción ha expirado",
        HttpStatus.BAD_REQUEST.value(), new Date());
    }

    @ExceptionHandler(HauntedHouseClosedException.class)
    public Error handlerHauntedHouseClosed(HauntedHouseClosedException ex) {
        return new Error(ex.getMessage(), description: "La Haunted House esta cerrada",
        HttpStatus.BAD_REQUEST.value(), new Date());
    }

    //Excepción para controlar el nivel de miedo (salta al ser un nivel mayor de 8)
    @ExceptionHandler(TooScaryException.class)
    public Error handlerTooScary(TooScaryException ex) {
        return new Error(ex.getMessage(), description: "La experiencia da demasiado miedo",
        HttpStatus.BAD_REQUEST.value(), new Date());
    }

    //Excepción para controlar que el nivel de miedo este entre 1-10
    @ExceptionHandler(InvalidScareLevelException.class)
    public Error handlerInvalidScareLevel(InvalidScareLevelException ex) {
        return new Error(ex.getMessage(), description: "Nivel inválido",
        HttpStatus.BAD_REQUEST.value(), new Date());
    }
```

```java
//Excepción para controlar que el id exista o sea válido
@ExceptionHandler(InvalidIdException.class)
public Error handlerInvalidId(InvalidIdException ex) {
    return new Error(ex.getMessage(), description: "Id inválido",
    HttpStatus.BAD_REQUEST.value(), new Date());
}

//Excepción para controlar que el id no sea un String
@ExceptionHandler(MethodArgumentTypeMismatchException.class)
public ResponseEntity<Error> handlerStringId(MethodArgumentTypeMismatchException ex) {
    Error errorPersonalizado = new Error(
        message: "El parámetro ID debe ser un número entero válido, no se admiten cadenas de texto.",
        description: "Tipo de parámetro inválido",
        HttpStatus.BAD_REQUEST.value(),
        new Date()
);
    return new ResponseEntity<>(errorPersonalizado, HttpStatus.BAD_REQUEST);
}

//Excepción para controlar que el usuario haya introducido el parámetro id (Ejemplo de error: /monster/ )
@ExceptionHandler(NoResourceFoundException.class)
public ResponseEntity<Error> handleMissingId(NoResourceFoundException ex) {

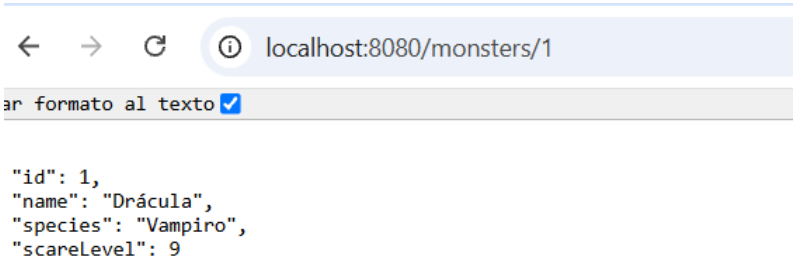
    Error errorResponseBody = new Error(
        message: "Falta el ID en la petición",
        description: "¡Cuidado! Te has olvidado de poner el ID del monstruo en la URL",
        HttpStatus.BAD_REQUEST.value(),
        new Date()
    );

    return new ResponseEntity<>(errorResponseBody, HttpStatus.BAD_REQUEST);
}
```

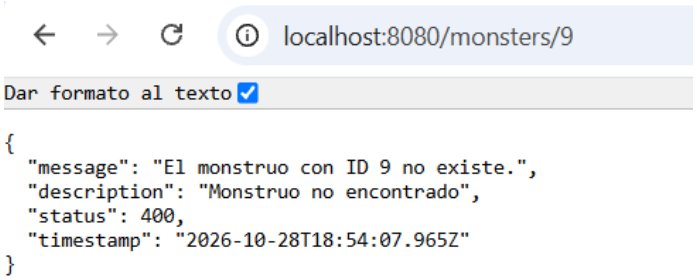
2. RESULTADOS OBTENIDOS

En este apartado se muestran las evidencias que demuestran el correcto funcionamiento de la aplicación o solución desarrollada.

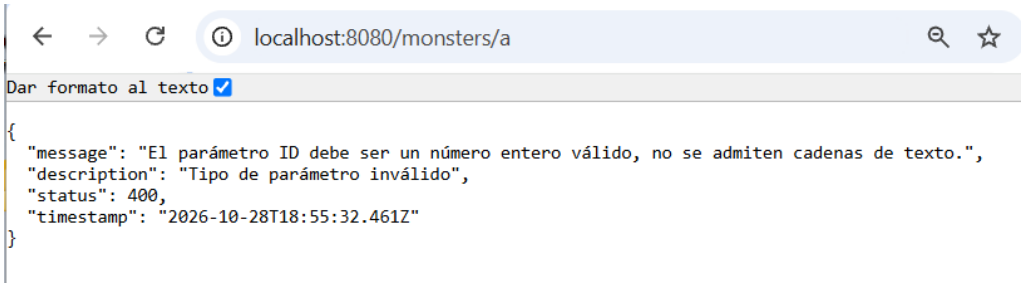
Además, se presentan los resultados alcanzados, las pruebas realizadas y las comprobaciones efectuadas para verificar que se cumplen los requisitos establecidos.



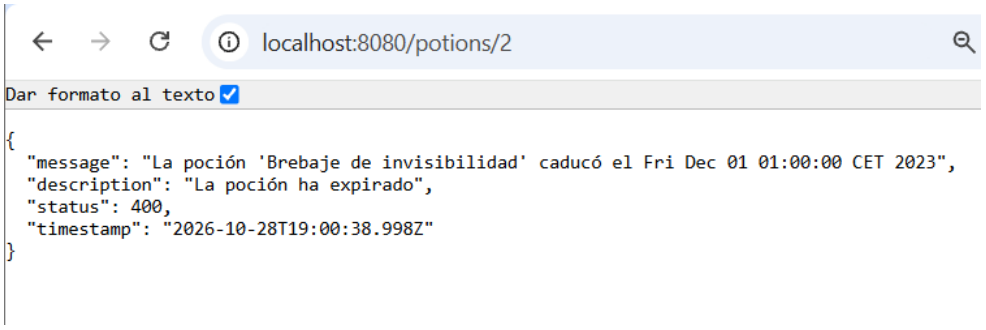
Monstruo con id 1.



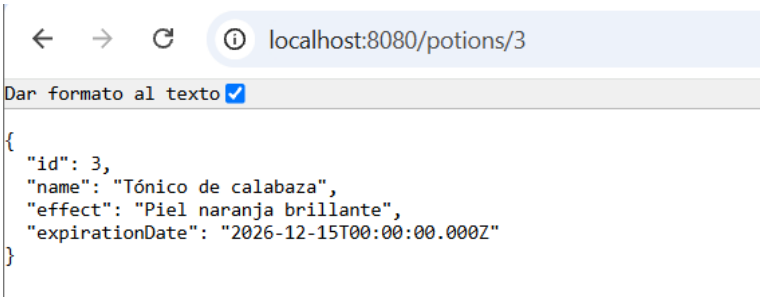
Error monstruo con id que no existe.



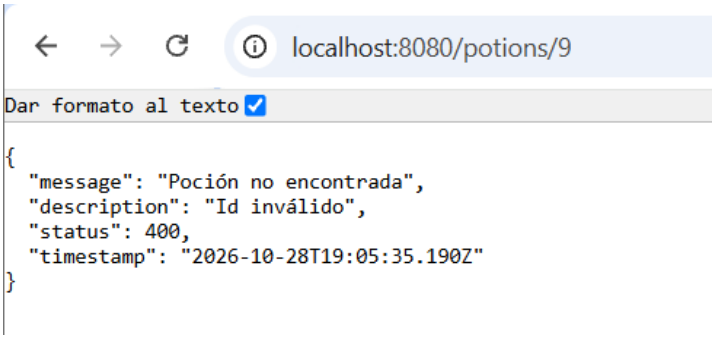
Error monstruo con id con letras en lugar de números.



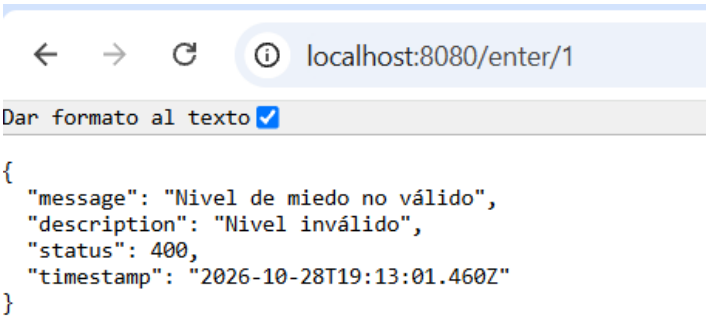
Error poción con id 2 caducada.



Poción id 3 la devuelve sin error.



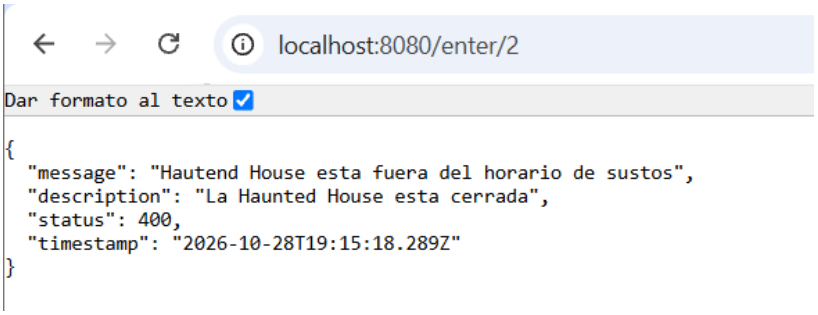
Error poción id no existe.



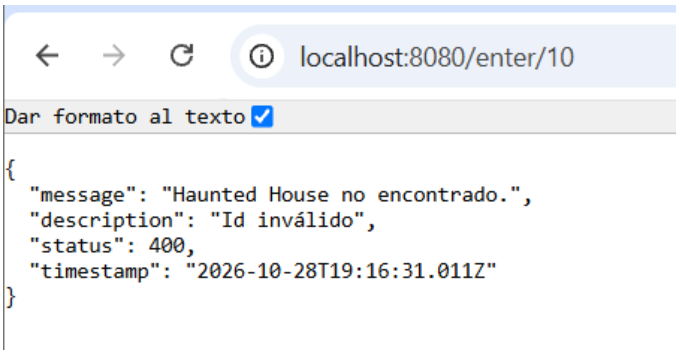
Error casa con scareLevel invalido.



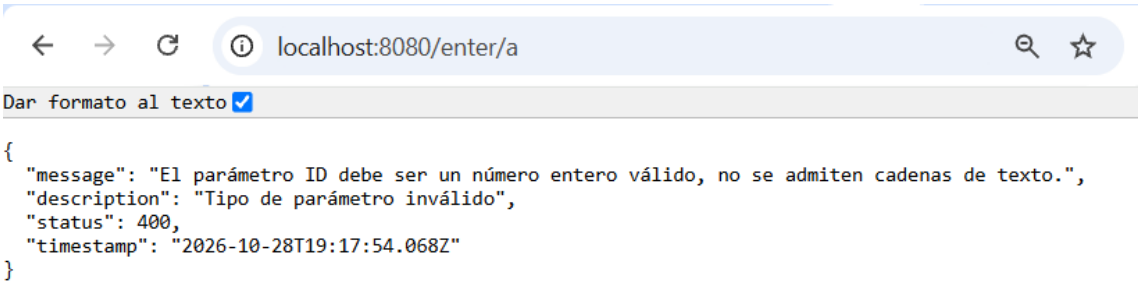
Casa abierta.



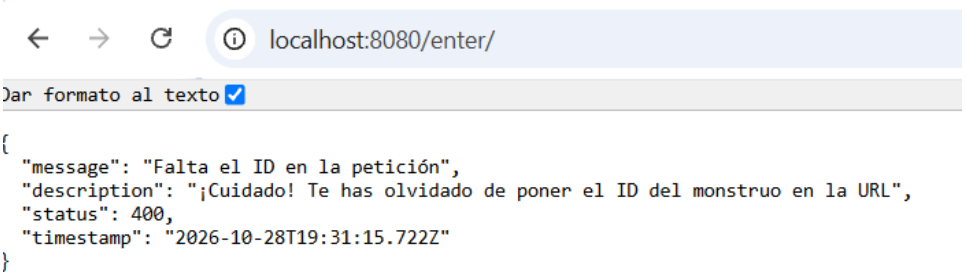
Error casa cerrada.



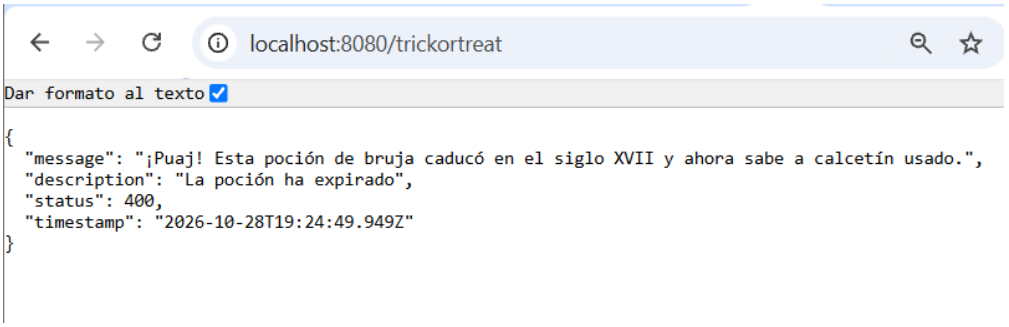
Error casa no existe.



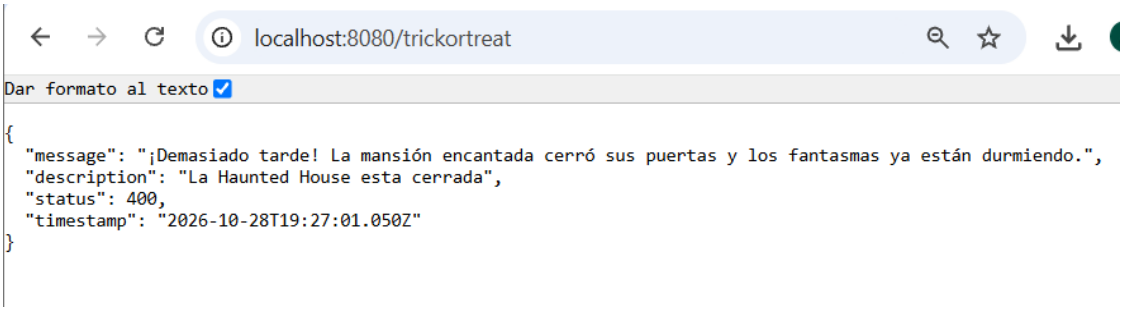
Error casa con id inválido de cadenas de texto.



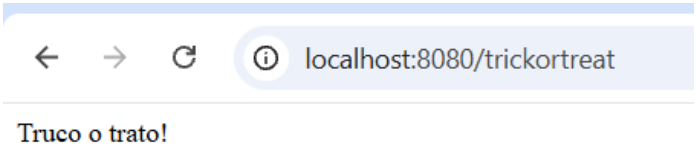
Error no introducir id



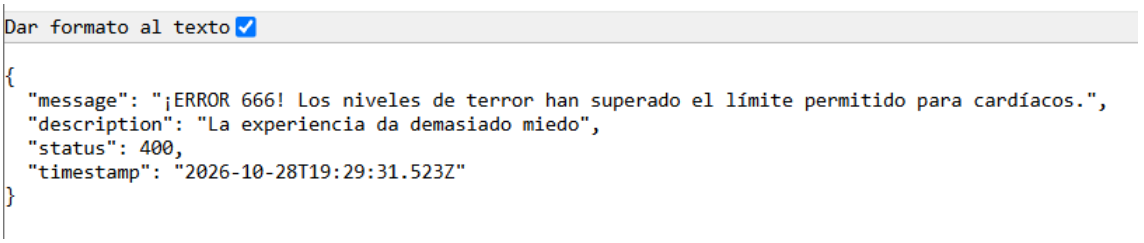
Controlador de "Truco o Trato" poción caducada.



Controlador de "Truco o Trato" casa cerrada.



Controlador de "Truco o Trato"



Controlador de "Truco o Trato" scareLevel alto.

3. DIFICULTADES ENCONTRADAS Y SOLUCIONES APLICADAS

Se han encontrado dificultades a la hora del manejo y control de los errores que puede producir el usuario a la hora de introducir en la URL un id, como por ejemplo que el id (de tipo Integer en el proyecto) no sea una cadena de caracteres o que lo deje con valor nulo. La solución que se ha aplicado para esta dificultad ha sido la investigación exhaustiva de excepciones ya existentes dentro del propio sistema que ya manejan este tipo de errores sin la necesidad de elaborar excepciones propias personalizadas y la lógica que conlleva.

Sigue siendo un poco confuso la conexión entre los repositorios y los archivos JSON para poder leer las instancias de los modelos. Se ha solucionado mirando ejercicios anteriores y accediendo a la práctica anterior para el manejo de ObjectMapper y Resource. Con estas herramientas se ha conseguido solventar esta dificultad.

4. CONCLUSIONES

Como conclusión final, el desarrollo de esta práctica ha permitido manejar el control de errores. Se han desarrollado excepciones personalizadas para cubrir la lógica solicitada y las mejoras opcionales gracias a **@RestControllerAdvice**. Al mismo tiempo, se han controlado errores propios de Spring Boot asegurando que cualquier fallo del sistema devuelva una respuesta controlada a través de excepciones intrínsecas en el sistema.

Además al finalizar esta práctica se entra de lleno en el ‘mundo de las excepciones’ que hasta ahora era un lugar que no se conocía y son muy útiles para la elaboración de código a la hora de desarrollar aplicaciones ya que son esenciales para su correcto funcionamiento.